

Tema 5: Colas de prioridad y árboles generales

5.1 La cola de prioridad

A diferencia de una cola normal FIFO (el primero en entrar es el primero en salir), una **cola de prioridad** atiende siempre al elemento **más grande o urgente** de todos los que esperan, sin importar cuándo ha entrado.

Si lo hacemos en un simple vector ordenado, el **push** es absurdamente lento $\mathcal{O}(N)$ al tener que mover todos los elementos. Y si la hacemos desordenada, el **pop** tardará $\mathcal{O}(N)$ en buscar dónde escondíamos el máximo. Necesitamos llevar el coste de ambas acciones a un corto $\mathcal{O}(\log N)$. ¿Cuál es la herramienta definitiva? El **binary heap**.

5.2 El binary heap

Es un **árbol binario completo** modelado dentro de un *simple vector plano*. La raíz siempre es el elemento máximo absoluto de toda la estructura.

El árbol ignora la posición 0. La raíz dominante vive coronada en la posición 1.

Para cualquier nodo en la posición i : - **Padre:** $i / 2$ - **Hijo Izquierdo:** $2 * i$ - **Hijo Derecho:** $2 * i + 1$

[Gràfic interactiu disponible a la versió web de l'apunt]

No hacen falta punteros ni reglas complejas, todo es aritmética fulgurante $\mathcal{O}(1)$.

push() - insertar y subir (flow up)

El elemento entra por la última posición del vector. Entonces se evalúa de abajo a arriba: *¿Soy más grande que mi jefe?*. Si es que sí, se intercambia (**swap**) de posición con él hacia arriba, iterando la subida hasta encontrar su lugar jerárquico.

[Simulació interactiva disponible a la web]

pop() - extraer y bajar (flow down)

El rey (posición 1) ha salido al ser procesado. Para sustituirlo, tomamos al “peón” (el último de todos en el vector) y lo plantamos en la casilla 1 de la raíz. Entonces se evalúa de arriba a abajo contra los dos hijos: *¿Cuál de mis dos nuevos súbditos inferiores es más grande que yo?*. Se intercambia la posición con el hijo **más grande**, iterando hacia abajo hasta el hueco que se merece.

[Simulació interactiva disponible a la web]

Resumen de tiempos heap: **top()** obtiene la raíz de inmediato en $\mathcal{O}(1)$. Las inserciones y limpiezas modifican la altitud del árbol, requiriendo solo saltos de coste $\mathcal{O}(\log N)$.

5.3 Customizando elementos: el operador >

A menudo tus elementos a examinar serán tuplas o personalizaciones como la propia `struct Persona`. Dado que le pides al Heap que *coloque el máximo arriba*, tendrás que dar reglas de juego (C++) donde redefines literalmente el operador MAYOR QUE (>) entre dos personas en el ámbito global.

```
struct Persona {
    string nombre;
    int prioridad;
};

// Sobrescribimos la lógica de "mayor que" de C++
bool operator>(const Persona& a, const Persona& b) {
    return a.prioridad > b.prioridad;
}
```

En el `main`, una vez hagamos `Heap<Persona> cola;`, ¡el directivo interno ya sabrá clasificar VIPs rápido por la edad o puntuación precargada!.

5.4 Árboles generales (Tree<T>)

Se acabó limitarse al cierre izquierdo/derecho. En un `Tree<T>`, un nodo Padre puede albergar un número arbitrario e infinito de nodos hijo: un “**vector**” literal de hijos.

Esto cambia toda la matriz de búsqueda. La etapa recursiva de llamar a `t.left()` y `t.right()` se modifica completamente adoptando un **bucle iterativo sobre los hijos** `t.child(i)`, pero el resto de artificios recursivos e inmutables permanecen exactos.

La clase tiene la siguiente organización visible en su parte pública: - `Tree()`: creador estándar vacío absoluto. - `Tree(const T& val)`: crea la coronación (hoja única base del valor sin hijos). - `Tree(const T& val, vector<Tree> hijos)`: crea la raíz asignando un valor e inyectando una ramificación múltiple de golpe gracias a un vector inicial precargado.

Consultoras prácticas para nuestra recursión: 1. `value()`: para leer el corazón de la raíz. 2. `num_children()`: para la condición de parada en el límite iterativo (N hijos). 3. `child(i)`: desplazamiento en el vector interno extrayendo cada nodo padre para reiniciar la llamada de salto.

Buscar profundamente en un general

Se itera la cantidad de hijos de 0 a límite-1. En caso de un acierto interior devolvemos el booleano a la cadena superior parando toda dispersión errática cerca de un $\mathcal{O}(N)$ salvador. No esperamos que todo acabe como en la antigua apelación de dos partes, al encontrar la clave cortocircuitamos la estrategia con un `True` contundente que desapila sin remordimientos hacia la raíz principal garantizando un rendimiento sólido e impecable para el sistema moderno de gestión de datos.

[Simulación interactiva disponible a la web]